

Lab2 实验报告

一、思考题

1、程序代码中的地址

- 请根据上述说明，回答问题：在编写的 C 程序中，指针变量中存储的地址被视为虚拟地址，还是物理地址？MIPS 汇编程序中 lw 和 sw 指令使用的地址被视为虚拟地址，还是物理地址？

1) 指针变量存储的是虚拟地址，现代操作系统都采用虚拟内存管理，程序运行在虚拟地址空间中，而不是直接访问物理内存。CPU 通过 MMU 将虚拟地址转为物理地址进而访存，因为映射关系不同，相同值的指针访问的实际物理内存也有可能不同。2) lw 和 sw 指令也是使用虚拟地址。

2、链表宏

- 从可重用性的角度，阐述用宏来实现链表的好处。

1) Java, C++ 等高级语言都有着泛型的身影，而 C 语言却没有，因此宏可以用来实现与泛型类似的功能，提高链表的可重用性。2) 宏可以实现跨项目复用，将链表的宏定义提取在一个单独的文件，只要在其他文件包含这个头文件就可以使用宏，而不用重复编写代码。3) 宏在预处理阶段展开，生成的代码直接嵌入调用处，无函数调用开销。

- 查看实验环境中的 /usr/include/sys/queue.h，了解其中单向链表与循环链表的实现，比较它们与本实验中使用的双向链表，分析三者插入与删除操作上的性能差异。

1) 单向链表：插入操作简单，只需要两行代码，将上一个结点的 next 指针指向新结点，将新结点的 next 指针指向下一个结点。删除操作复杂，需要遍历找到前驱节点，将前驱节点的 next 指针指向下一个结点，时间复杂度 $O(n)$ ；2) 循环链表：插入与删除操作和单向链表的复杂度类似，不过无需判断头结点和尾结点的特殊情况；3) 双向链表：插入与删除操作复杂度均为 $O(1)$ ，性能出色，但需要额外的内存来支付代价。

3、Page_list 结构体

- 请阅读 include/queue.h 以及 include/pmap.h，将 Page_list 的结构梳理清楚，选择正确的展开结构。

```
// A
struct Page_list {
    struct {
        struct {
            struct Page *le_next;
            struct Page *le_prev;
        } pp_link;
        u_short pp_ref;
    } *lh_first;
}

// B
struct Page_list {
    struct {
        struct {
```

```

        structPage *le_next;
        structPage **le_prev;
    }pp_link;
    u_shortpp_ref;
} lh_first;
}
// C
structPage_list{
    struct {
        struct{
            structPage *le_next;
            structPage **le_prev;
        }pp_link;
        u_shortpp_ref;
    }*lh_first;
}

```

```

//Pmap.h
LIST_HEAD(Page_list, Page);
typedef LIST_ENTRY(Page) Page_LIST_entry_t;

struct Page {
    Page_LIST_entry_t pp_link;
    u_short pp_ref;
};
//Queue.h
#define LIST_HEAD(name, type) \
    struct name { \
        struct type *lh_first; \
    }

#define LIST_ENTRY(type) \
    struct { \
        struct type *le_next; \
        struct type **le_prev; \
    }

```

拼接起来即可发现C正确

4、地址空间

- 请阅读上面有关TLB的描述，从虚拟内存和多进程操作系统的实现角度，阐述ASID的必要性。

在不同进程中，同一虚拟地址在不同地址空间中映射的物理地址有可能不同，ASID 作为地址空间标识符，通过为每个进程分配唯一的 ASID，TLB 可以同时缓存多个进程的地址映射。在地址转换时，CPU 不仅检查虚拟页号，还检查 ASID 是否匹配，从而避免不同进程之间的映射冲突。而缓存多个进程的映射也可以不用再进程切换时清空 TLB，提高 TLB 的命中率，减少 TLB 刷新开销。

- 请阅读 MIPS 4Kc 文档《MIPS32® 4K™ Processor Core Family Software User’sManual》的 Section 3.3.1 与 Section 3.4，结合 ASID 段的位数，说明 4Kc 中可容纳不同的地址空间的最大数量。

Table 3-6 TLB Tag Entry Fields

Field Name	Description																								
PageMask[24:13]	Page Mask Value. The Page Mask defines the page size by masking the appropriate VPN2 bits from being involved in a comparison. It is also used to determine which address bit is used to make the even-odd page (PFN0-PFN1) determination. See the table below. <table><tr><th>PageMask[11:0]</th><th>Page Size</th><th>Even/Odd Bank Select Bit</th></tr><tr><td>0000_0000_0000</td><td>4KB</td><td>VAddr[12]</td></tr><tr><td>0000_0000_0011</td><td>16KB</td><td>VAddr[14]</td></tr><tr><td>0000_0000_1111</td><td>64KB</td><td>VAddr[16]</td></tr><tr><td>0000_0011_1111</td><td>256KB</td><td>VAddr[18]</td></tr><tr><td>0000_1111_1111</td><td>1MB</td><td>VAddr[20]</td></tr><tr><td>0011_1111_1111</td><td>4MB</td><td>VAddr[22]</td></tr><tr><td>1111_1111_1111</td><td>16MB</td><td>VAddr[24]</td></tr></table> The PageMask column above show all the legal values for PageMask. Because each pair of bits can only have the same value, the physical entry in the JTLB will only save a compressed version of the PageMask using only 6 bits. This is however transparent to software, which will always work with a 12 bit field.	PageMask[11:0]	Page Size	Even/Odd Bank Select Bit	0000_0000_0000	4KB	VAddr[12]	0000_0000_0011	16KB	VAddr[14]	0000_0000_1111	64KB	VAddr[16]	0000_0011_1111	256KB	VAddr[18]	0000_1111_1111	1MB	VAddr[20]	0011_1111_1111	4MB	VAddr[22]	1111_1111_1111	16MB	VAddr[24]
PageMask[11:0]	Page Size	Even/Odd Bank Select Bit																							
0000_0000_0000	4KB	VAddr[12]																							
0000_0000_0011	16KB	VAddr[14]																							
0000_0000_1111	64KB	VAddr[16]																							
0000_0011_1111	256KB	VAddr[18]																							
0000_1111_1111	1MB	VAddr[20]																							
0011_1111_1111	4MB	VAddr[22]																							
1111_1111_1111	16MB	VAddr[24]																							
VPN2[31:13]	Virtual Page Number divided by 2. This field contains the upper bits of the virtual page number. Because it represents a pair of TLB pages, it is divided by 2. Bits 31:25 are always included in the TLB lookup comparison. Bits 24:13 are included depending on the page size, defined by PageMask.																								
G	Global Bit. When set, indicates that this entry is global to all processes and/or threads and thus disables inclusion of the ASID in the comparison.																								
ASID[7:0]	Address Space Identifier. Identifies which process or thread this TLB entry is associated with.																								

// 原文

The address space identifier (ASID) helps to reduce the frequency of TLB flushing on a context switch. The existence of the ASID allows multiple processes to exist in both the TLB and instruction caches. The ASID value is stored in the EntryHi register and is compared to the ASID value of each entry.

如图，MIPS 4Kc 的 TLB 条目包含 8 位 ASID 字段，因此可支持 256 个不同的地址空间。

5、tlb_invalidate

- tlb_invalidate和tlb_out的调用关系？

tlb_invalidate 是高层 TLB 管理 C 语言函数；tlb_out 是底层汇编函数，由 tlb_invalidate 调用，执行具体的 TLB 无效化操作（通过 MIPS 的 tlbw 和 tlbwi 指令）。

- 请用一句话概括tlb_invalidate的作用。

tlb_invalidate 负责在虚拟地址映射发生变化（如页表修改、进程切换）时，清除指定虚拟地址对应的 TLB 条目。

- 逐行解释tlb_out中的汇编代码。

```
LEAF(tlb_out)           # 声明一个叶子函数（无嵌套调用）
.set noreorder          # 禁用指令重排序
    mfc0    t0, CP0_ENTRYHI    # 保存当前 EntryHi 到 t0 寄存器
    mtc0    a0, CP0_ENTRYHI    # 将参数 va 写入 EntryHi
    nop                                # 延迟槽填充

    tlbp                                # 查找 TLB 中匹配 va 的条目，结果存入 Index
    nop                                # 延迟槽填充
    mfc0    t1, CP0_INDEX    # 读取 Index 值到 t1 寄存器
.set reorder          # 启用指令重排序

    bltz    t1, NO_SUCH_ENTRY    # 如果 Index < 0（未命中），跳转到恢复阶段
.set noreorder          # 禁用指令重排序

    mtc0    zero, CP0_ENTRYHI    # 清零 EntryHi（准备无效化条目）
    mtc0    zero, CP0_ENTRYLO0    # 清零 EntryLo0（低半页物理地址）
    mtc0    zero, CP0_ENTRYLO1    # 清零 EntryLo1（高半页物理地址）
    nop
    tlbwi                                # 将清零后的 EntryHi/Lo 写回 TLB
.set reorder

NO_SUCH_ENTRY:
    mtc0    t0, CP0_ENTRYHI    # 恢复原始 EntryHi 值
    j       ra                # 返回调用函数
END(tlb_out)
```

6、三级页表自映射

- 在现代的 64 位系统中，提供了 64 位的字长，但实际上不是 64 位页式存储系统。假设在 64 位系统中采用三级页表机制，页面大小 4KB。由于 64 位系统中字长为 8B，且页目录也占用一页，因此页目录中有 512 个页目录项，因此每级页表都需要 9 位。因此在 64 位系统下，总共需要 $3 \times 9 + 12 = 39$ 位就可以实现三级页表机制，并不需要 64 位。现考虑上述 39 位的三级页式存储系统，虚拟地址空间为 512GB，若三级页表的基址为 PTbase，请计算：（1）三级页表页目录的基址。（2）映射到页目录自身的页目录项（自映射）。

1) 三级页表页目录的基址： $\$(PTbase \gg 21) \ll 3 + (PTbase \gg 12) \ll 3 + PTbase$
 2) 映射到页目录自身的页目录项（自映射）： $\$(PTbase \gg 30) \ll 3 + (PTbase \gg 21) \ll 3 + (PTbase \gg 12) \ll 3 + PTbase$

7、函数调用与CPU访存流程

- 请结合 Lab2 开始的 CPU 访存流程与下图中的 Lab2 用户函数部分，尝试将函数调用与 CPU 访存流程对应起来，思考函数调用与 CPU 访存流程的关系。

函数作为数据存储在内存中，在函数调用前，CPU准备参数，将需要用到的参数压入栈中。准备完毕后，CPU将函数地址（函数名）压入栈中，根据函数地址访问在内存中的函数，函数执行过程中，CPU根据函数中的指令，访问内存中的数据，进行计算。函数执行完毕后，CPU将返回地址压入栈中，跳转回调用函数处，继续执行。可以发现函数调用通过“指令流 + 数据流”驱动 CPU 的访存行为。

8、其他体系结构的内存管理

- 简单了解并叙述X86体系结构中的内存管理机制，比较X86和MIPS 在内存管理上的区别。

X86 采用 段页式内存管理，结合分段和分页机制，分段机制通过段寄存器和段描述符表将逻辑地址转为线性地址；分页机制支持多级页表，将线性地址转为物理地址。X86 的分段机制增加了复杂性，但提供更细粒度的内存保护；MIPS 仅依赖分页，更简单高效。MIPS 的 TLB 支持 ASID，减少进程切换时的 TLB 刷新；X86 需手动管理 TLB。

- 简单了解并叙述RISC-V 中的内存管理机制，比较RISC-V 与 MIPS 在内存管理上的区别。

RISC-V 采用纯分页机制，支持多级页表，TLB管理采用 RVWMO（弱内存序）。RISC-V 的页表项包含 ASID 字段，支持多进程共享同一页表。RISC-V 的分页机制更灵活，支持更大地址空间；而 MIPS 的页表结构较固定。RISC-V 的 TLB 管理更现代化，可选 ASID；MIPS 需依赖固定 ASID 机制。

- 简单了解并叙述LoongArch 中的内存管理机制，比较 LoongArch 与 MIPS 在内存管理上的区别。

LoongArch 采用类 RISC-V 的分页机制，但优化了 TLB 和地址转换。TLB 支持大页和 ASID，内存访问控制：采用平面内存模型，简化地址转换。而 MIPS TLB 更传统，分段式更简洁。

二、难点分析

本次实验第一个难点我认为是理解并补全`queue.h`中关于双向链表的功能，不同于我们在大一数据结构学习的链表，这次实验中给出的链表采用宏来实现，是BSD风格链表，并且结点的前驱指针指向前一个指针的后继指针的地址，虽然说起来很绕，让理解也变得十分困难，但是这个设计让结点的添加与删除操作的时间复杂度都降到了 $O(1)$ ，且插入头部、中间、尾部的逻辑相同，简化了代码。

本次实验第二个难点就是就是完成与页面（page）相关的函数，因为一开始我没有搞懂一些变量在代码中的地址是物理地址还是虚拟地址，对链表宏的理解也不深，所以费了很长一段时间才解决。

本次实验第三个难点就是实现虚拟内存管理，补全一系列函数。因为这部分内容我在理论课上就没有完全理解，所以到实际运用中就更加吃力了。同时也不清楚虚拟地址与物理地址的转换，之后还是结合指导书、往届学长博客和开源仓库的答案我才大致梳理清楚了整体的实现过程。

三、实验体会

本次实验的难度相较于上一次有了很大的提升，正式进入了操作系统的主要工作，我对内存管理也有了更深入的理解。在完成作业过程中，我也大量使用了大语言模型，让他帮我解释`qmap.h`和`queue.h`的部分代码，不然面对大量复杂的代码，我完全无从下手，只能抓瞎。虽然在实现虚拟内存管理的部分，有部分练习我没能自己完全做出来，最后求助于实验的开源仓库，但通过这次实验，我对内存管理有了更深入的理解，也锻炼了代码阅读能力。希望能顺利通过本周的Lab2课上实验，在之后的Lab中也能攻坚克难，坚持到最后的胜利！

四、原创说明

- 本文部分内容在LLM（大语言模型）的协助下完成

